

MedGraph

Guia de entendimento do projeto

Tech Challenge — Fase 3 · Pós-Tech 8IADT · Hospital Vida Plena (cenário fictício)

Este documento explica, em linguagem direta, o que o MedGraph é, como as peças se encaixam, o que cada parte do código faz, como executar o projeto do começo ao fim e como qualquer pessoa que receba o repositório gera o próprio link do Colab.

1. Do que se trata

O enunciado da Fase 3 pede um assistente virtual médico treinado com dados de um hospital, capaz de auxiliar em condutas clínicas, responder dúvidas do corpo médico e sugerir procedimentos com base em protocolos internos — coordenando fluxos de decisão automatizados e seguros.

O MedGraph atende a isso combinando três coisas que costumam aparecer separadas:

- uma **LLM ajustada** por fine-tuning para o formato de resposta que queremos;
- um **fluxo de decisão** explícito, com etapas nomeadas e auditáveis;
- **limites de atuação** verificados por código, e não apenas pedidos no prompt.

1.1 De onde vem o nome

Med de médico. **Graph** de grafo — a estrutura de dados que organiza o fluxo de decisão. O nome descreve a arquitetura: em vez de uma única chamada à LLM que faz tudo de uma vez, o projeto é um grafo de etapas nomeadas, cada uma com uma responsabilidade, ligadas por caminhos que podem se desviar.

É essa forma que torna o sistema auditável. Não se pergunta "por que a IA respondeu isso?", e sim "por quais nós esta consulta passou, e o que cada um decidiu?" — pergunta que tem resposta em arquivo.

1.2 O princípio que rege tudo

O assistente nunca prescreve. Ele apresenta evidência, aponta a fonte de cada afirmação e devolve a decisão ao médico responsável.

Isso não é postura de marketing: é o requisito 3 do enunciado, implementado nos guardrails, no nó de validação humana e nos testes automatizados. Quando o risco de uma resposta passa de um limiar configurado, a execução simplesmente **para** e espera um médico.

1.3 O que ele faz na prática

Um médico pergunta, opcionalmente vinculando um paciente:

"Qual a conduta antibiótica inicial para sepse de foco pulmonar neste paciente?"

O sistema então limpa a pergunta de dados identificáveis, descobre que tipo de pergunta é, consulta o prontuário estruturado, busca evidência científica e protocolos internos, pede à LLM uma resposta ancorada apenas nesse material, confere a resposta contra regras clínicas de segurança, exige citação de fonte, calcula um escore de risco e — se for o caso — retém a resposta para validação humana. Tudo isso fica registrado numa trilha de auditoria consultável.

No exemplo acima, o paciente tem alergia grave a penicilina. A resposta sugere uma cefalosporina. O sistema reconhece que **ceftriaxona é betalactâmico, a mesma classe da penicilina**, marca conflito crítico, e a execução para.

2. O vocabulário do projeto

Esta seção existe para que o resto do documento possa ser lido sem consultar nada por fora. Se você já conhece os termos, pule para a seção seguinte.

2.1 Sobre modelos de linguagem

LLM (Large Language Model). Um modelo de linguagem de grande porte — o tipo de programa por trás do ChatGPT. Ele foi treinado prevendo a próxima palavra em enormes quantidades de texto, e nesse processo acabou aprendendo gramática, fatos e padrões de raciocínio. Aqui usamos um modelo de **3 bilhões de parâmetros**, pequeno para os padrões atuais, que roda num computador comum.

Parâmetro. Cada um dos números internos do modelo, ajustados durante o treino. "3 bilhões de parâmetros" é o tamanho do modelo.

Token. A unidade em que o modelo enxerga o texto: um pedaço de palavra. Uma frase de 20 palavras costuma virar 25 a 30 tokens. Os custos e os limites de tamanho são todos contados em tokens.

Prompt. O texto que se envia ao modelo. Neste projeto os prompts ficam todos em `chains/prompts.py` — um prompt espalhado pelo código é um prompt que ninguém revisa.

Inferência. O ato de usar o modelo para gerar uma resposta, por oposição a treiná-lo.

2.2 Sobre o ajuste do modelo

Fine-tuning (ajuste fino). Pegar um modelo já treinado e continuar treinando com dados específicos, para que ele aprenda um comportamento novo. Não é ensinar medicina ao modelo: é ensinar o **formato** de resposta que queremos — decisão na primeira linha, citação da fonte no fim.

SFT (Supervised Fine-Tuning). O tipo de ajuste usado aqui: mostram-se pares de pergunta e resposta ideal, e o modelo aprende a produzir a segunda a partir da primeira.

Adapter. O arquivo que sai do treino: apenas as peças novas, sem o modelo inteiro. No nosso caso, cerca de 50 MB — pequeno o bastante para ficar versionado no Git ao lado do código que o produziu.

Época. Uma passada completa por todos os exemplos de treino.

Passo (step). Uma atualização dos parâmetros. Um treino tem centenas deles.

Perda (loss). O quanto o modelo está errando. É o número que precisa cair ao longo do treino; se a perda de treino cai e a de validação sobe, o modelo está decorando em vez de aprender.

2.3 Sobre a infraestrutura

GPU. A placa de vídeo, que faz as multiplicações de matriz em paralelo. Sem ela, treinar um modelo de 3 bilhões de parâmetros levaria dias.

VRAM. A memória da GPU. É o recurso escasso: determina se o treino cabe ou não.

CUDA. A tecnologia da NVIDIA que permite usar a GPU para cálculo. Bibliotecas de treino dependem dela — e é por isso que o fine-tuning não roda num Mac.

Google Colab. Um serviço do Google que dá acesso gratuito a uma GPU pelo navegador, com um limite diário de uso.

Hugging Face. O repositório público de onde se baixam modelos e conjuntos de dados.

Modelo gated. Modelo cujo download exige aceitar uma licença. O Llama 3.2 é um deles.

Ollama. Um programa que roda modelos de linguagem na sua própria máquina e os expõe como um serviço local. É o que serve o modelo depois de pronto, sem custo por consulta.

2.4 Sobre a recuperação de evidência

RAG (Retrieval-Augmented Generation). Geração aumentada por recuperação. Em vez de confiar no que o modelo memorizou, busca-se o material relevante primeiro e pede-se que ele responda **apenas** com base nesse material. É o que permite citar a fonte de cada afirmação — e o que reduz a chance de o modelo inventar.

Embedding. A representação de um texto como uma lista de números, construída de modo que textos com sentido parecido fiquem próximos. É o que permite buscar por significado, e não por palavra exata.

Vector store. O banco que guarda esses vetores e responde "quais textos mais se parecem com este?". Aqui usamos o **FAISS**, que roda local e é salvo em disco.

2.5 Sobre o fluxo e a segurança

LangChain. Biblioteca que padroniza a conversa com modelos de linguagem — montagem de prompt, chamada, tratamento da resposta.

LangGraph. Extensão do LangChain para fluxos em forma de grafo, com estado compartilhado, caminhos condicionais e possibilidade de **pausar a execução** à espera de uma pessoa. É essa última capacidade que sustenta a validação médica.

Guardrail. Uma verificação que limita o que o sistema pode fazer ou dizer. Aqui há um na entrada (limpa a pergunta e recusa o que está fora de escopo) e um na saída (exige citação, confere se as fontes existem, marca posologia).

PII (Personally Identifiable Information). Dados que identificam uma pessoa: nome, CPF, telefone, prontuário. O projeto os remove antes de qualquer texto chegar ao disco.

Trilha de auditoria. O registro, evento por evento, do que aconteceu em cada consulta — qual nó rodou, quanto demorou, o que decidiu. É o que permite reconstruir uma resposta meses depois.

3. Como funciona, em uma passagem

Uma pergunta atravessa um grafo de **14 nós**. Cada nó recebe um estado, devolve o estado modificado, e é automaticamente registrado na auditoria.

#	Nó	O que faz
1	guardrail_entrada	Remove identificadores e recusa pedidos fora de escopo
2	responder_recusa	Caminho alternativo quando a entrada é recusada
3	classificar_intencao	Dúvida clínica, consulta ao paciente, conduta, resumo...
4	consultar_prontuario	Alergias, medicações, exames, comorbidades
5	recuperar_evidencia	Busca semântica em artigos e protocolos
6	raciocinio_clinico	A LLM responde, ancorada no contexto recuperado
7	regras_clinicas	Alergias, interações, valores críticos, função renal
8	guardrail_saida	Exige citação, verifica fontes, marca posologia
9	reescrever	Nova tentativa quando a saída é reprovada
10	degradar_resposta	Resposta mínima segura quando as tentativas se esgotam
11	triagem_risco	Calcula o escore de risco da resposta
12	aguardar_validacao	Retém a resposta e espera um médico
13	emitir_alertas	Publica os alertas para a equipe
14	montar_resposta	Monta o texto final com fontes e disclaimer

Os nós 2, 9, 10 e 12 são desvios: só entram quando algo exige. O caminho feliz passa pelos demais.

3.1 Onde ficam as decisões de rota

O arquivo `grafo/rotas.py` concentra as quatro decisões condicionais do fluxo: depois do guardrail de entrada, depois da classificação, depois do guardrail de saída e depois da triagem de risco. Separá-las dos nós mantém cada nó com uma responsabilidade só.

4. Arquitetura em camadas



Cada dependência abaixo do grafo é substituível sem tocar no fluxo — o grafo conhece interfaces, não implementações.

4.1 Decisões técnicas e o porquê de cada uma

Decisão	Razão
Fine-tuning por QLoRA no Colab	9 GB de VRAM em vez de 48; adapter de 50 MB em vez de 6 GB
Modelo servido pelo Ollama	Roda offline, custo zero por consulta, código LangChain padrão
Embeddings locais (multilingual-e5-small)	Cobre inglês dos artigos e português dos protocolos, sem custo
FAISS	Vector store persistido em disco, sem serviço externo
Políticas em YAML	Governança clínica não deveria exigir leitura de Python
Auditoria por contextvars	Os nós mantêm assinatura limpa, sem carregar trace_id

5. Estrutura de pastas

fia_tech3/	
config/	
settings.py	Configuracao central, le o .env e valida
politicas.yaml	Limites de atuacao, em formato declarativo
src/medgraph/	
requisitos.py	Catalogo dos requisitos do enunciado
logging_config.py	Logging em tres destinos
auditoria.py	Trilha, trace por consulta, @instrumentar
dados/	Download, anonimizacao, curadoria, corpus
finetune/	Dataset de treino e utilidades do Colab
avaliacao/	Metricas, comparativos e graficos
llm/	Provedores de modelo e controle de custo
rag/	Indexacao vetorial e recuperacao com fontes
prontuario/	Acesso a base estruturada de pacientes
chains/	Pipelines LangChain
guardrails/	Guardrails de entrada e saida, regras clinicas
grafo/	Fluxo LangGraph
ui/	Painel Streamlit
data/	Dados brutos, processados, sinteticos, indices
models/	Adapter LoRA e modelo quantizado
notebooks/colab/	Notebooks de fine-tuning e exportacao
scripts/	Pontos de entrada de linha de comando
tests/	Testes automatizados
docs/	Relatorio, diagramas, graficos, rastreabilidade
logs/	Trilha de auditoria e traces

6. O que cada módulo faz

6.1 Fundação

`config/settings.py` — classe `Settings`, baseada em `Pydantic`. Lê o `.env`, valida os valores e deriva todos os caminhos do projeto. Um erro de configuração falha aqui, na largada, e não vinte minutos depois.

`requisitos.py` — classe `Requisito` e o `CATALOGO` com os 13 requisitos do enunciado. Cada docstring do código cita a tag correspondente, e a matriz de rastreabilidade é gerada varrendo essas tags.

`logging_config.py` — classes `FormatadorJSONL` e `FiltroApenasAuditoria`. Configura três destinos simultâneos: console colorido, arquivo de texto rotativo, e a trilha JSONL formal.

`auditoria.py` — o coração da rastreabilidade. Classes `TipoEvento`, `Desfecho`, `EventoAuditoria` e `TrilhaAuditoria`. O decorador `@instrumentar` registra automaticamente início, fim, duração e o delta de estado de cada nó. A alternativa — repetir blocos de log em cada nó — falharia no primeiro esquecimento, e um nó sem rastro invalidaria a garantia de auditabilidade.

6.2 Dados

`dados/anonimizador.py` — classes `TipoPII`, `Politica`, `Achado` e `Anonimizador`. Detecta e trata dados pessoais. Cuidado central do projeto: um anonimizador que apaga valor laboratorial entrega dado limpo e clinicamente inútil.

`dados/baixar_pubmedqa.py` — obtém o PubMedQA do Hugging Face.

`dados/curadoria.py` — classe `RelatorioCuradoria` e as funções `higienizar`, `curar`, `dividir_estratificado` e `balancear_por_rotulo`. A divisão estratificada preserva a proporção das classes; o balanceamento corrige a classe rara.

`dados/construir_banco.py` — monta a base SQLite de prontuários sintéticos.

6.3 Fine-tuning

`finetune/preparar_dataset_sft.py` — monta o dataset supervisionado a partir do PubMedQA curado, dos protocolos e do FAQ.

`finetune/colab_utils.py` — tudo o que o notebook do Colab precisa, em módulo Python testável: `verificar_gpu`, `config_quantizacao` (a parte "Q" do QLoRA, em NF4 com dupla quantização), `config_lora`, `montar_configuracao_sft`, `montar_treinador`, `alinhar_precisao_dos_adaptadores`, `grafico_de_perda` e `salvar_metadados` (o cartão de treino — sem ele o adapter seria um binário sem procedência).

O notebook é deliberadamente magro: ele orquestra, não implementa. Lógica dentro de célula de notebook não tem teste, não tem lint e não aparece em diff legível.

6.4 Modelo e custo

llm/provider.py — classes `CallbackCusto` e `ChatEco`, funções `obter_llm` e `obter_llm_com_fallback`. Abstrai qual modelo responde: o ajustado no Ollama, a API paga, ou um eco determinístico para testes.

llm/custo.py — classes `RegistroUso`, `ContadorCusto` e `OrcamentoExcedidoError`. Contabiliza tokens e dólares por chamada e **bloqueia** chamadas pagas ao atingir o teto da sessão.

6.5 Recuperação de evidência

rag/indexar.py — classes `EstatisticasIndice` e `EmbeddingsComPrefixo`. Constrói o índice FAISS. O prefixo existe porque os modelos E5 exigem marcar o texto como consulta ou como documento.

rag/recuperador.py — classes `Trecho` e `Recuperador`. Devolve trechos já etiquetados com o marcador de fonte, o que torna a citação verificável.

6.6 Prontuário

prontuario/modelos.py — as entidades do domínio: `Paciente`, `Alergia`, `Medicacao`, `Exame`, `Comorbidade`, `SinalVital`, `Evolucao`.

prontuario/repositorio.py — classe `RepositorioProntuarios`, a única porta de acesso à base estruturada.

6.7 Pipelines LangChain

chains/prompts.py — todos os textos de prompt num lugar só.

chains/chain_triagem.py — classe `ResultadoTriagem`. Classifica a intenção da pergunta.

chains/chain_rag.py — monta a mensagem com o contexto recuperado e obtém a resposta ancorada.

6.8 Segurança e validação

guardrails/politicas.py — classes `PadraoBloqueio` e `Politicas`. Carrega o `politicas.yaml` e normaliza o texto antes de comparar, para que acentuação não contorne um bloqueio.

guardrails/entrada.py — classe `ResultadoEntrada`. Limpa a pergunta e recusa o que está fora de escopo.

guardrails/regras_clinicas.py — o módulo mais denso do projeto. Classes `Interacao`, `Severidade`, `Achado` e `ResultadoVerificacao`; verificações de alergias, interações medicamentosas, valores laboratoriais críticos, função renal e populações especiais. A função `em_contexto_de_evitacao` distingue "evitar penicilina" de "prescrever penicilina" — sem ela, o sistema alertava justamente quando o assistente acertava.

`guardrails/saida.py` — classes `Falha` e `ResultadoSaida`. Exige citação de fonte, verifica se as fontes citadas existem, marca posologia e acrescenta o disclaimer.

6.9 O grafo

`grafo/estado.py` — `EstadoClinico`, o dicionário tipado que atravessa todos os nós.

`grafo/nos.py` — as 14 funções de nó, cada uma decorada com `@instrumentar`.

`grafo/rotas.py` — as quatro decisões condicionais.

`grafo/construir.py` — monta e compila o grafo.

`grafo/executar.py` — classe `Consulta` e as funções `consultar`, `validar` e `consultas_pendentes`. É a fachada que o painel e os scripts usam.

`grafo/diagrama.py` — gera o diagrama do fluxo em PNG, Mermaid e ASCII.

6.10 Avaliação e interface

`avaliacao/metricas.py` — classe `ResultadoAvaliacao`, extração da decisão, baseline da classe majoritária e a tabela comparativa.

`avaliacao/avaliar.py` e `avaliacao/graficos.py` — executam o comparativo entre os sistemas e desenharam os gráficos.

`ui/app_streamlit.py` e `ui/componentes.py` — o painel visual, com abas para consulta, prontuário, fontes, alertas, trilha do grafo e validação médica.

7. O fine-tuning explicado

Esta é a parte do projeto que costuma parecer mais opaca, e ela tem uma lógica simples por trás.

7.1 O problema

Queremos que o modelo responda **sempre no mesmo formato**: a decisão na primeira linha, o raciocínio em seguida, a citação da fonte no fim. Um modelo genérico responde bem, mas cada vez de um jeito — e um formato instável quebra tudo o que vem depois, porque os guardrails precisam encontrar a citação para verificá-la.

Ajustar o modelo resolve isso. Mas treinar um modelo de 3 bilhões de parâmetros do jeito tradicional exige guardar, além dos pesos, os gradientes e os estados do otimizador: cerca de **48 GB de memória de vídeo**. Nenhuma GPU gratuita chega perto disso — a T4 do Colab tem 16 GB.

7.2 A ideia do LoRA

LoRA significa *Low-Rank Adaptation*, adaptação de baixo posto. A intuição é esta: para ensinar um formato de resposta a um modelo que já sabe português e já leu literatura médica, não é preciso mexer nos 3 bilhões de números. A mudança necessária é pequena e tem estrutura.

Então congela-se o modelo inteiro e adicionam-se, ao lado de certas camadas, **duas matrizes finas** cujo produto representa o ajuste. Treinam-se apenas elas — cerca de **24 milhões de parâmetros, 0,7% do total**.

	Fine-tuning completo	LoRA
Parâmetros treinados	3,2 bilhões	24 milhões
Memória necessária	~48 GB	~16 GB
Tamanho do resultado	~6 GB	~50 MB

Os 50 MB são a razão de o resultado do treino caber no Git, versionado junto com o código que o produziu.

7.3 O "Q" de QLoRA

Falta um passo. Mesmo congelado, o modelo base precisa estar na memória — e em precisão normal ele ocupa ~6 GB, o que ainda aperta.

Quantização é reduzir a precisão numérica dos pesos. Em vez de 16 bits por número, usam-se **4 bits**. O modelo cai de ~6 GB para ~2 GB, com perda de qualidade pequena.

O formato usado é o **NF4** (*NormalFloat4*), proposto no artigo do QLoRA: otimizado para números que seguem distribuição normal, que é o caso dos pesos de uma rede treinada. Perde menos que o int4 comum. Há ainda a **dupla quantização**, que comprime as próprias constantes de quantização e economiza mais uns 0,4 bit por parâmetro.

QLoRA = Quantized LoRA: o modelo base entra quantizado em 4 bits e congelado; os adaptadores treinam por cima, em precisão maior. O cálculo acontece numa precisão mais alta do que o armazenamento — os pesos são desquantizados no momento da multiplicação.

É exatamente por isso que o treino é lento numa GPU modesta: cada multiplicação de matriz paga o custo de desquantizar antes de calcular. Não é defeito de configuração; é o preço de caber.

7.4 Onde os adaptadores são aplicados

Nas **sete projeções** de cada bloco do modelo: as quatro da atenção (q , k , v , o) e as três do MLP ($gate$, up , $down$).

Muitos tutoriais aplicam LoRA apenas em q_proj e v_proj . Isso rende menos quando a tarefa muda o **estilo** da resposta — e é o nosso caso: queremos que o modelo passe a responder num formato rígido.

7.5 O que sai do treino

Um diretório com o adapter, o tokenizador, a curva de perda e o `cartao_de_treino.json`. Esse cartão guarda qual modelo base foi usado, com quais hiperparâmetros, em qual GPU, com quais versões de biblioteca, em quanto tempo e com que perda final. **Sem ele, o adapter seria um binário sem procedência.**

7.6 Do adapter ao modelo que responde

O adapter sozinho não é utilizável por um servidor comum. Faltam três passos, que o segundo notebook executa:

Fusão (merge). Somar as matrizes do adapter aos pesos do modelo base, produzindo um modelo único e completo. É aqui que importa fundir na arquitetura **certa**: um adapter treinado sobre um modelo e fundido em outro não falha — só responde pior, silenciosamente.

Conversão para GGUF. O GGUF é o formato de arquivo que o `llama.cpp` e o Ollama entendem, desenhado para rodar modelos em hardware comum. A conversão também quantiza de novo, agora em **Q4_K_M** — um nível de compressão que equilibra tamanho e qualidade.

Publicação e registro. O arquivo vai para o Hugging Face, e na máquina local o Ollama o registra sob um nome. A partir daí, `OLLAMA_MODEL=medgraph` no `.env` faz todo o restante do projeto passar a conversar com o modelo ajustado — sem mudar uma linha de código.

7.7 Um detalhe que importa: o chat template

Cada família de modelos marca o início e o fim de cada fala com tokens especiais próprios. Esse padrão chama-se **chat template**.

O `Modelfile` do Ollama, neste projeto, **não fixa** um template: usa o que vem embutido no próprio GGUF. Se ele trouxesse o formato de um modelo escrito à mão — como chegou a trazer numa versão inicial —, trocar de modelo base degradaria as respostas em silêncio, porque o modelo veria

uma sequência de tokens diferente da que aprendeu.

8. Execução passo a passo

Este documento descreve **toda** a execução do projeto, célula a célula, do primeiro clone até o vídeo de entrega. Ele existe para ser seguido sem conhecimento prévio do projeto e para servir de base a quem precise explicar o procedimento a outra pessoa.

Os tempos citados foram **medidos** numa T4 do Colab gratuito com Llama-3.2-3B-Instruct, e não estimados. Onde um número for aproximado, está dito.

Convenção: "célula 7" significa a sétima célula de código. O Colab

intercala células de texto, que não contam. Cada seção numerada do notebook

corresponde a uma célula de código.

8.1 Visão geral

Etapas	Onde	Tempo
0. Preparar a máquina	local	~10 min
1. Treino (notebook 01)	Colab	30 min a 6 h, conforme a configuração
2. Exportação (notebook 02)	Colab	~25 min
3. Registro e avaliação	local	~20 min
4. Vídeo	local	~1 h de gravação

As etapas 1 e 2 exigem GPU com CUDA e por isso rodam no Colab. Todo o resto roda no seu computador.

8.2 Etapa 0 — Preparar a máquina local

0.1 Clonar e instalar

```
git clone https://github.com/alexandreccarmo/fia_tech3.git
cd fia_tech3
make setup
```

O `make setup` cria o `.venv` com Python 3.12, instala as dependências, gera o `.env` a partir do `.env.example` e roda a suíte de testes.

0.2 Conferir o ambiente

```
make ambiente
```

Apresenta uma tabela item a item. **AVISO** em artefatos de etapas seguintes é esperado; **FALHA** precisa ser resolvido antes de continuar.

0.3 Preparar os dados

```
make dados
make indexar
```

O primeiro baixa o PubMedQA, anonimiza, cura e gera o corpus hospitalar sintético. O segundo constrói o índice FAISS. Ambos já vêm versionados no repositório — rode apenas se quiser reproduzir do zero.

0.4 Contas necessárias

Conta	Para quê
Google	Acessar o Colab
Hugging Face	Baixar o modelo base e publicar o GGUF

No Hugging Face, crie um token de **escrita** em *Settings* → *Access Tokens* → *Create new token* → *Write*. Ele será usado no segundo notebook.

0.5 Aceitar a licença do modelo

O Llama 3.2 é *gated*: exige aceite. Acesse huggingface.co/meta-llama/Llama-3.2-3B-Instruct, preencha o formulário e envie.

A aprovação não é instantânea. Pode levar de minutos a horas, e não há como consultar a fila. O aceite fica vinculado à **conta do Hugging Face**, não à conta Google — trocar de conta Google não o invalida.

Se não quiser esperar, o notebook traz `Qwen/Qwen2.5-3B-Instruct` como alternativa aberta, comentada na célula 7. Nesse caso **use o mesmo modelo no notebook 02**.

8.3 Como se chega ao notebook no Colab

Os links abaixo abrem os notebooks direto do GitHub. Vale entender como eles são formados, porque quem trabalhar sobre uma cópia própria vai precisar montar os seus.

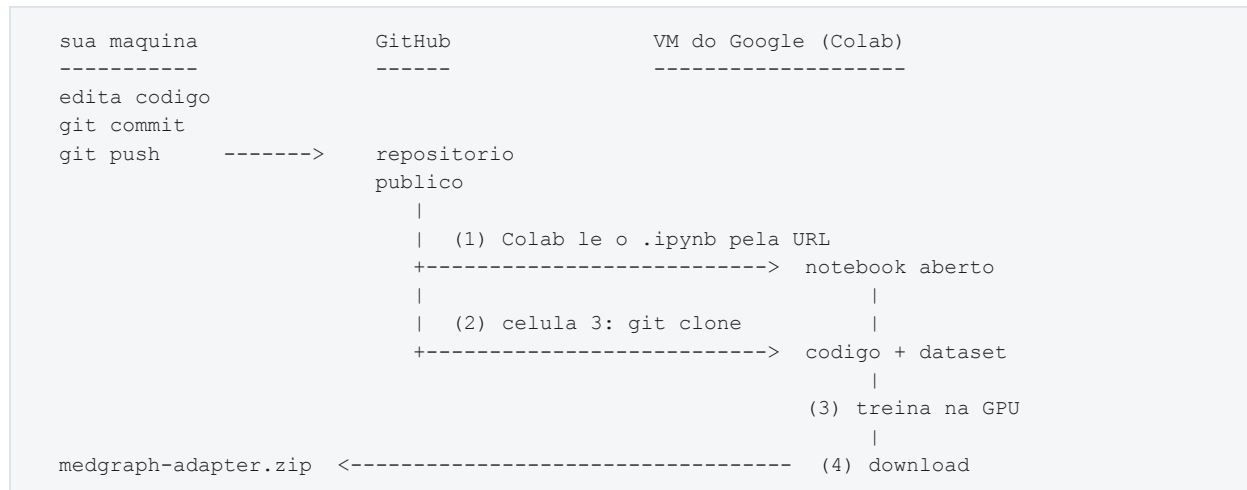
Não há geração nem integração. O Colab tem um carregador de GitHub que funciona por padrão de endereço: é a URL do arquivo no GitHub com `github.com/` trocado por `colab.research.google.com/github/`.

```
https://github.com/USUARIO/REPO/blob/BRANCH/CAMINHO.ipynb
https://colab.research.google.com/github/USUARIO/REPO/blob/BRANCH/CAMINHO.ipynb
```

O Colab baixa aquele `.ipynb` — que é apenas JSON versionado como qualquer outro arquivo — e o abre no editor. Funciona sem login porque o repositório é público.

O notebook, sozinho, não traz o código do projeto. Quem traz é a célula 3, que executa `git clone` do repositório dentro da máquina virtual e acrescenta `src/` ao caminho de importação do Python. É por isso que a célula seguinte consegue fazer `from medgraph.finetune import colab_utils`. O dataset de treino vem no mesmo clone, porque também está versionado.

O caminho é de mão única: o Colab lê do GitHub e nunca escreve de volta sozinho. O resultado do treino retorna como download manual do `.zip`.



Para uma cópia própria do projeto

Quem for modificar o projeto precisa que a célula 3 clone **o repositório dele** — senão o Colab treinaria com o código de outra pessoa.

1. Publique a cópia no GitHub como repositório **público**.
2. Na célula 3, aponte a variável `REPO` para ela:

```
REPO = "https://github.com/NOVO_USUARIO/NOVO_REPO.git"
```

E ajuste o caminho do clone, se o nome do diretório mudar.

3. Monte o link trocando as partes do endereço:

```
https://colab.research.google.com/github/NOVO_USUARIO/NOVO_REPO/blob/main/notebooks/colab/01_finetune_qlora_pubmedqa.ipynb
```

4. Repita para o `02_exportar_gguf.ipynb`.

Sem montar a URL à mão

Dentro do Colab: **Arquivo** → **Abrir notebook** → **aba GitHub**, digite `usuario/repositorio` e escolha o notebook na lista.

Se o repositório for privado

A mesma aba GitHub tem a opção **Incluir repositórios privados**, que pede autorização da conta. Mas o link direto deixa de funcionar para quem não tiver acesso, e a célula do `git clone` também falharia — ela roda sem credenciais dentro da VM. É por isso que este projeto é público: é o que torna "treinar" equivalente a "abrir um link".

8.4 Etapa 1 — Notebook 01: o treino

Abra:

```
https://colab.research.google.com/github/alexandreccarmo/fia_tech3/blob/main/notebooks/colab/01_finetune_qlora_pubmedqa.ipynb
```

Ative a GPU: **Ambiente de execução** → **Alterar o tipo de ambiente de execução** → **T4 GPU** → **Salvar**.

Célula 1 — Verifica a GPU

Instantâneo. Deve imprimir `Tesla T4`.

Se disser que não há GPU, são duas causas: o runtime está sem acelerador (resolva no menu acima), ou a cota diária acabou. Para distinguir, rode `!nvidia-smi`: se o comando não existir e o menu já mostrar T4, é cota.

Célula 2 — Instala as bibliotecas

~3 minutos.

Depois dela, reinicie a sessão: *Ambiente de execução* → *Reiniciar sessão*. E continue **da célula 3**, não do começo. Este é o ponto onde mais gente tropeça.

Célula 3 — Clona o repositório

~30 segundos. Traz o código e o dataset de treino para dentro da VM.

Ela só clona **se a pasta ainda não existir**. Para forçar código atualizado: `!rm -rf /content/fia_tech3` antes.

Célula 4 — Confere GPU e versões

Instantâneo. O aviso "GPU sem suporte a bfloat16 (esperado na T4)" é normal.

Célula 5 — Login no Hugging Face

~1 minuto. Ela imprime um **código de dispositivo**. Abra hf.co/oauth/device em outra aba, já logado na conta que tem o aceite, digite o código **com o hífen** e autorize.

O código expira em poucos minutos; se perder o prazo, reexecute a célula.

Célula 6 — Carrega o dataset

~20 segundos. Mostra um exemplo completo, para conferência.

Ajuste opcional — reduzir o treino

Se a configuração padrão não couber na sua cota (ver *Quanto tempo leva*, abaixo), insira **uma célula nova entre a 6 e a 7** e cole:

```
EXEMPLOS = 2000

colab_utils.CONFIG_PADRAO["num_train_epochs"] = 1
colab_utils.CONFIG_PADRAO["logging_steps"] = 5
colab_utils.CONFIG_PADRAO["eval_steps"] = 25
colab_utils.CONFIG_PADRAO["save_steps"] = 25
colab_utils.CONFIG_PADRAO["save_total_limit"] = 2

dados["train"] = dados["train"].shuffle(seed=42).select(range(min(EXEMPLOS, len(dados["train"]))))
dados["validation"] = dados["validation"].select(range(min(64, len(dados["validation"]))))

lote = (colab_utils.CONFIG_PADRAO["per_device_train_batch_size"]
        * colab_utils.CONFIG_PADRAO["gradient_accumulation_steps"])
print(f"{len(dados['train'])} exemplos | ~{len(dados['train'])//lote} passos "
      f"| ~{len(dados['train'])//lote*45/60:.0f} min")
```

Duas regras ao mexer nesses valores:

- `save_steps` precisa ser múltiplo de `eval_steps`, senão o `load_best_model_at_end` recusa a configuração;
- reduza `eval_steps` junto com o treino. Com o padrão de 100, um treino de 26 passos não avaliaria nenhuma vez, e a curva da célula 11 sairia sem a linha de validação.

O `shuffle(seed=42)` é determinístico: a mesma amostra sai em toda execução, o que torna o treino reproduzível e permite retomar de checkpoint.

Célula 7 — Baixa e quantiza o modelo base

3 a 8 minutos. Baixa ~6 GB e quantiza para 4 bits em memória.

É aqui que se escolhe o modelo. A linha ativa deve ser a que você quer:

```
MODELO_BASE = "meta-llama/Llama-3.2-3B-Instruct"    # gated — exige aceite
# MODELO_BASE = "Qwen/Qwen2.5-3B-Instruct"         # aberto
```

Ao terminar, imprime:

```
Parâmetros: 3.21 B
VRAM ocupada: 2.09 GB
```

Confira a contagem de parâmetros: 3,21 B é o Llama-3.2-3B; 3,09 B é o Qwen2.5-3B. Se não for o modelo que você escolheu, pare — o resto da execução seria sobre a arquitetura errada. A VRAM bem abaixo dos 6 GB baixados confirma que a quantização funcionou.

Falha com `GatedRepoError` ou 403 significa que a conta não tem o aceite.

Célula 8 — Configura o LoRA

Instantâneo. Prepara o modelo e informa quantos parâmetros treinarão.

Ela **não** chama `get_peft_model` de propósito: quem aplica o LoRA é o `SFTTrainer`, na célula seguinte. Aplicar duas vezes empilha adaptadores e derruba o treino com um erro que não menciona PEFT.

Célula 9 — Monta o treinador

Instantâneo, mas é a célula de decisões.

Ligue o Drive se o treino for durar mais de meia hora:

```
USAR_DRIVE = True
```

Custa ~1 minuto de autorização e ~1,5 GB do seu Drive, e é o que faz os checkpoints sobreviverem à reciclagem da VM, a quedas de internet e a quedas de energia. Numa rodada longa, vale sempre.

Ao final ela imprime **quantos passos** serão dados. Confira que bate com o que você configurou — se você reduziu o dataset e ela imprime o número cheio, o ajuste não foi aplicado.

A mensagem `Argumentos nao suportados ... IGNORADOS: [...]` é informativa: a camada de compatibilidade removeu o que a versão instalada do `trl` recusa.

Célula 10 — O treino

~45 segundos por passo numa T4. Multiplique pelo número de passos que a célula 9 imprimiu.

A barra mostra `[passo/total tempo_decorrido < tempo_restante]`. O total cobre **todas** as épocas, não uma.

A tabela de perdas aparece a cada `eval_steps`. O que se espera: `Training Loss` e `Validation Loss` caindo juntas. Validação subindo enquanto treino cai é sobreajuste.

Mantenha a aba aberta. E leia *Quando algo dá errado*, abaixo, antes de reagir a qualquer desconexão.

Célula 11 — Curva de perda

Instantâneo. Desenha treino e validação no mesmo gráfico e informa a redução percentual da perda de validação.

Célula 12 — Testa a adesão ao formato

~1 minuto. Gera resposta para 5 exemplos do conjunto de teste e verifica duas coisas em cada uma: se começa com `Decisão: yes|no|maybe` e se cita `[E1]`.

Espera-se 4 ou 5 de 5. Menos que isso significa treino insuficiente — mas só julgue por aqui se o treino foi completo. Num treino curto de dezenas de passos, formato baixo é esperado e não indica erro.

Célula 13 — Grava o adapter

~10 segundos. Salva em `models/adapters/medgraph-llama32-3b-lora`: os pesos do adapter, o tokenizador, a curva de perda e o `cartao_de_treino.json` — que registra modelo base, hiperparâmetros, versões, GPU, duração e perdas.

Sem esse cartão o adapter seria um binário sem procedência.

Célula 14 — Baixa o .zip

~20 segundos. **Guarde este arquivo.** Ele é o produto do treino.

8.5 Etapa 2 — Notebook 02: exportação

Abra:

```
https://colab.research.google.com/github/alexandreccarmo/fia\_tech3/blob/main/notebooks/colab/02\_exportar\_gguf.ipynb
```

Se rodar na **mesma sessão** do notebook 01, o adapter já está em disco. Em sessão nova, a célula 3 pede o upload do `.zip`.

Célula 1 — Ambiente

Informa se há GPU. **Nada aqui exige CUDA**, mas use a **T4** mesmo assim.

A fusão da célula 5 carrega o modelo base em float16 — ~6,4 GB — e cria cópias durante o merge, contra os ~12,7 GB de RAM do Colab gratuito. A folga é estreita, e estourar a memória **derruba a sessão inteira**, levando junto o upload do adapter e o login. Com a GPU, o modelo vai para a VRAM e a RAM fica livre.

A conversão do `llama.cpp`, mais adiante, é CPU de qualquer forma.

Célula 2 — Dependências e repositório

~3 minutos. Instala, remove o `torchao` e clona.

Os avisos em vermelho do `pip` nesta célula são esperados: ele reclama que `google-ai-generativelanguage`, `ydf` e `grpcio-status` — pacotes de fábrica do Colab — querem uma versão mais antiga do `protobuf`. Nenhum é usado aqui. O sinal de que a célula terminou é a linha `Resolving deltas: 100%, do git clone`.

A remoção do `torchao` não é capricho: o Colab traz a versão 0.10 pré-instalada e o `peft` recente exige 0.16 ou superior. A incompatibilidade não aparece aqui — ela estoura na célula 5, dentro de `PeftModel.from_pretrained`, com um `ImportError` que fala de `torchao` e não menciona o adapter. Este notebook não usa a biblioteca: a quantização é do `llama.cpp` e a fusão é em float16.

Célula 3 — O adapter

Localiza o adapter. Se não encontrar, abre o seletor de upload para o `medgraph-adapter.zip`.

Ela imprime o **cartão de treino**: modelo base, exemplos, duração e perdas. Confira o `modelo_base` — é a última chance de perceber que se está fundindo na arquitetura errada.

Célula 4 — Autenticação

Mesmo fluxo de código de dispositivo do notebook 01. Aqui o token precisa ser de **escrita**, porque o notebook publica no Hub.

Célula 5 — Fusão

Ajuste o `MODELO_BASE` para o mesmo do notebook 01. É o erro mais caro possível neste ponto: fundir num modelo diferente não falha — gera um GGUF que carrega, responde, e responde mal.

A fusão soma as matrizes do adapter aos pesos do modelo base e produz um modelo único e completo. Roda em CPU e consome bastante RAM.

Célula 6 — llama.cpp

Clona e compila o conversor e o quantizador. Alguns minutos.

Célula 7 — Conversão e quantização

Converte o modelo fundido para GGUF em f16 e depois quantiza para **Q4_K_M** — o nível que equilibra tamanho e qualidade.

Célula 8 — Publicação

Cria o repositório no Hugging Face e envia o GGUF junto com o `Modelfile`. Ao final imprime o nome exato do repositório e a linha para o seu `.env`:

```
REPO_GGUF_HF=seu-usuario/medgraph-llama32-3b-gguf
```

Anote esse valor.

Célula 9 — Alternativa

Download direto do GGUF, caso você prefira não publicar no Hub.

8.6 Etapa 3 — De volta à máquina

3.1 Descompactar o adapter

```
cd ~/Desktop/FIAP/projeto/tech_challenge_fiap3/fia_tech3
unzip ~/Downloads/medgraph-adapter.zip -d models/adapters/
```

Vêm junto o `cartao_de_treino.json` e a `curva_de_perda.png` — os dois artefatos que o vídeo mostra no bloco de fine-tuning. Para deixar o gráfico junto dos demais:

```
cp models/adapters/medgraph-llama32-3b-lora/curva_de_perda.png docs/graficos/
```

3.2 Ajustar o `.env`

```
REPO_GGUF_HF=seu-usuario/medgraph-llama32-3b-gguf
HF_TOKEN=hf_...
OLLAMA_MODEL=medgraph
```

Só troque `OLLAMA_MODEL` para `medgraph` **depois** de registrar o modelo no passo seguinte. Antes disso, o valor correto continua sendo `medgraph-base`.

3.3 Registrar no Ollama

```
make modelo -- --ajustado
```

Baixa o GGUF do Hub e registra no Ollama. O separador `--` é obrigatório: é ele que faz a flag chegar ao script.

Confira:

```
ollama list
ollama run medgraph "Qual a conduta inicial na suspeita de sepse?"
```

3.4 Reavaliar

```
make avaliar
make relatorio
```

A tabela comparativa passa a ter a coluna do modelo ajustado ao lado do modelo base e do `gpt-4o-mini`. O relatório técnico é regenerado com os números novos e com os dados do cartão de treino.

Para uma passada rápida de conferência: `make avaliar -- --rapido` (30 casos).

3.5 Conferir tudo

```
make testes
make lint
make rastreabilidade
```

8.7 Etapa 4 — O vídeo

```
make app
```

Abre o painel Streamlit, que é o cenário da maior parte da gravação. O roteiro cronometrado, com a divisão entre os integrantes, está em `roteiro_video.md`.

Dos 8 blocos do roteiro, **6 não dependem do modelo ajustado** — inclusive o Bloco 5, a demonstração ao vivo, que é o núcleo. Eles podem ser gravados a qualquer momento, com o `medgraph-base`. Só os blocos 3 (fine-tuning) e 7 (avaliação) precisam do treino concluído.

8.8 Quanto tempo leva o treino

Medido: **~45 segundos por passo** numa T4 gratuita.

O número de passos é $\text{exemplos} \times \text{épocas} \div 16$. Com o dataset completo de 3.871 exemplos e 2 épocas, são 484 passos — **quase 6 horas**, mais do que a cota diária do plano gratuito, que fica entre 3 e 4 horas.

Configuração	Passos	Tempo
Padrão (2 épocas, 3.871 ex.)	484	~6 h
1 época	242	~3 h
1 época, 2.000 exemplos	125	~1 h 35
1 época, 800 exemplos	50	~40 min

Reduzir o dataset é preferível a reduzir o `max_seq_length`: os exemplos têm ~900 tokens em média, e cortar para 512 trancaria a resposta esperada em metade deles.

8.9 Quando algo dá errado

A conexão caiu

Não reinicie a sessão por reflexo. Perder o navegador não é perder o treino: a VM continua executando na nuvem, indiferente ao seu computador.

1. Recarregue a página. Isso reconecta a interface ao mesmo kernel. 2. Se o "Conectando" não sair, feche **todas** as abas do Colab e reabra em janela anônima — extensões de bloqueio derrubam o websocket com frequência. 3. Confirme em *Ambiente de execução* → *Gerenciar sessões* se a sessão continua ativa. "Última execução: há 0 minuto" significa que ela está trabalhando agora.

Nunca abra duas abas do mesmo notebook. Elas disputam a sessão e é uma causa comum de "Conectando" eterno.

Acompanhar o treino sem o Colab

Se você ligou o `USAR_DRIVE`, abra drive.google.com e navegue até `Meu Drive / medgraph / saida_treino`. As pastas `checkpoint-N` e suas horas de modificação mostram o progresso **sem depender de o Colab conectar**.

Só aparecem os últimos checkpoints porque `save_total_limit` limita quantos ficam em disco. Não é perda de dado.

Retomar de um checkpoint

Rode as células 3 a 9 — **incluindo o bloco de ajuste**, se você usou um `—`, com o mesmo valor de `USAR_DRIVE`. Confira que a célula 9 imprime o mesmo número de passos de antes. Então, na célula 10:

```
resultado = treinador.train(resume_from_checkpoint=True)
```

Recuperar o adapter sem rodar a célula 13

Cada checkpoint contém os pesos treinados. Se a sessão morreu depois do treino mas antes de salvar, o resultado **não se perdeu** — as células 11 a 14 apenas formatam e empacotam o que o checkpoint já tem.

No Drive, clique com o botão direito na pasta do checkpoint mais avançado e escolha **Fazer download**. Depois, na sua máquina:

```
make recuperar-adapter -- --checkpoint ~/Downloads/checkpoint-125
```

O comando copia o adapter e o tokenizador para `models/adapters/medgraph-llama32-3b-lora`, redesenha a curva de perda a partir do `trainer_state.json` e grava o cartão de treino. Aceita tanto a pasta de um checkpoint quanto a que contém vários — nesse caso escolhe o de maior número de passos.

Passe também `--modelo-base`, que o notebook de exportação lê:

```
make recuperar-adapter -- --checkpoint ~/Downloads/checkpoint-125 \
  --modelo-base meta-llama/Llama-3.2-3B-Instruct
```

O cartão gerado por essa via registra explicitamente que GPU, versões de biblioteca e duração **não foram capturadas** — esses dados só existiam na sessão do Colab. Um cartão de procedência que adivinha é pior do que um cartão incompleto.

O que se perde nesse caminho é o teste de formato da célula 12. Ele pode ser refeito na máquina depois de registrar o modelo no Ollama.

A cota de GPU acabou

O Colab não recusa a conexão: conecta em CPU e mantém "T4 GPU" no menu, o que faz parecer problema de configuração. Confirme com `!nvidia-smi`.

Não há o que consertar. A cota volta em algumas horas. Alternativas: outra conta Google (a cota é por conta, e o aceite do Hugging Face continua valendo), ou o Colab Pro.

Erros mais comuns

Sintoma	Causa	O que fazer
Nenhuma GPU disponível	Sem acelerador, ou cota	<code>!nvidia-smi</code> distingue
<code>GatedRepoError / 403</code> na célula 7	Conta sem aceite do Llama	Aceite, ou use o Qwen
<code>ModuleNotFoundError</code> na célula 4	Sessão não reiniciada após a 2	Reinicie e siga da 3
<code>CUDA out of memory</code>	Sequência longa demais	Antes da 9: <code>colab_utils.CONFIG_PADRAO["max_seq_length"] = 768</code>
<code>save_steps</code> não é múltiplo de <code>eval_steps</code>	Ajuste manual incoerente	Igual os dois valores
Formato baixo na célula 12	Treino insuficiente	Mais épocas ou mais exemplos
<code>FileNotFoundError: nvidia-smi</code> na célula 1 do notebook 02	Runtime sem GPU, que aqui é o esperado	Corrigido; em notebook antigo, pule a célula
<code>ImportError: incompatible version of torchao</code> na célula 5 do notebook 02	<code>torchao 0.10</code> do Colab contra o mínimo do <code>peft</code>	<code>%pip uninstall -y -q torchao</code> e reexecute
Sessão morre na célula 5 do notebook 02	RAM esgotada durante a fusão	Troque para T4 GPU e use <code>device_map="auto"</code>

8.10 Checklist de conclusão

- `models/adapters/medgraph-llama32-3b-lora/` com adapter e cartão de treino
- GGUF publicado no Hugging Face Hub
- `.env` com `REPO_GGUF_HF` e `OLLAMA_MODEL=medgraph`
- `ollama list` mostrando `medgraph`
- `make avaliar` com a coluna do modelo ajustado
- `make relatorio` regenerado
- `make testes` e `make lint` limpos
- Vídeo de até 15 minutos gravado (REQ-E4)
- Repositório com tudo commitado e enviado

9. Requisitos do enunciado

O projeto cataloga cada exigência com um código, citado nas docstrings do código que a atende. A matriz completa é gerada automaticamente por `make rastreabilidade`.

Código	Exigência
REQ-1	Fine-tuning de um modelo LLM com dados do hospital
REQ-1a	Preparo dos dados: preprocessing, anonimização e curadoria
REQ-2	Pipeline LangChain integrando a LLM customizada
REQ-2a	Consultas a base de dados estruturada
REQ-2b	Respostas contextualizadas com dados do paciente
REQ-3a	Limites de atuação para evitar sugestões impróprias
REQ-3b	Logging detalhado para rastreamento e auditoria
REQ-3c	Explainability: indicar a fonte da informação
REQ-4	Projeto modularizado com instruções completas no README
REQ-E1	Código-fonte com os fluxos do LangGraph
REQ-E2	Dataset anonimizado ou dados sintéticos
REQ-E3	Relatório técnico detalhado
REQ-E4	Vídeo de até 15 minutos

10. Testes e honestidade dos números

A suíte roda em poucos segundos e cobre configuração, catálogo de requisitos, logging, auditoria, cálculo de custo, trava de orçamento, regras clínicas e a consistência do arquivo de políticas — incluindo a verificação de que conduta terapêutica **sempre** exige validação humana.

Há ainda uma categoria de teste incomum: `tests/test_documentacao.py` verifica as afirmações numéricas da documentação contra o código. Ele nasceu de um erro real — o grafo foi planejado com doze nós, ganhou mais dois durante a implementação, e cinco arquivos continuaram afirmando "doze". Nenhum teste falhou, nenhum linter reclamou, e o erro sobreviveu a várias revisões porque documentação não é executada.

O relatório técnico segue o mesmo princípio: a narrativa fica em `docs/relatorio_base.md` e os números são lidos dos artefatos que o pipeline produziu. Um relatório com números digitados à mão começa correto e envelhece errado. Este guia também é gerado, por `make guia`, a partir de `docs/guia_do_projeto.md`.

11. Nota de transparência

Nenhum dado real de paciente é utilizado. Prontuários, protocolos e documentos do Hospital Vida Plena são gerados por script, com nomes fictícios. Ainda assim, o pipeline de anonimização é aplicado sobre eles, demonstrando a técnica exigida no enunciado e garantindo que o mesmo código funcionaria sobre dados reais.

Este é um projeto acadêmico. Não foi submetido a comitê de ética, não passou por validação clínica e não deve ser utilizado em assistência a pacientes.